

2.3.3 Algorithmic Strategy: Abstraction

ایبسٹریکشن (Abstraction) کا مطلب ہے غیر ضروری تفصیلات کو چھپا کر صرف ضروری باتوں پر توجہ دینا۔

CS XI

Chapter 2: Computational Thinking & Algorithm

Lecture 05 of 11

2.3.3 Algorithmic Strategy: Abstraction

LEARN IT

Abstraction means hiding unnecessary features and focusing only on the essential ones needed to solve a problem. It reduces complexity by ignoring what is not important (the **noise**).

When you apply abstraction you sort details into three groups:

- **Necessary steps** — the core logic that never changes.
- **Changes based on the situation** — these become **variables** (inputs) in the algorithm.
- **Abstraction items (noise)** — details you can ignore completely.

WORKED EXAMPLE

Example: applying abstraction to "Making Tea"

To make an algorithm for tea, we separate the necessary steps from the situational changes and the noise.

1. Necessary steps (core logic)	2. Changes based on situation (variables)	3. Abstraction items (ignore / noise)
1. Boil water 2. Add tea leaves/bag 3. Steep for a few minutes 4. Pour into cup 5. Serve	Ingredients (black, green, milk) Additions (sugar, lemon) Tools (stove vs kettle)	Brand of kettle Cup colour or design Where tea is made (kitchen, office)

The core abstracted logic, independent of cup colour or kettle brand:

Boil water → Add tea → Steep → Pour → Serve

This one algorithm now works for green tea in an office kettle and for milk tea on a kitchen stove. The middle column simply becomes inputs: `tea_type`, `addition`, `tool`.

Computers use abstraction everywhere. When you call `print("Hello")` in Python, you do not think about how the screen lights up each pixel — those details are hidden from you. A map of Karachi shows roads and landmarks but hides every tree and car: it is an abstraction of the city.

REMEMBER IT

Abstraction = "**Keep the skeleton, drop the decoration.**" Ask: "If I remove this detail, does the solution still work?" If yes, it is noise.

COMMON MISTAKE

Do not confuse abstraction with decomposition. Decomposition *splits* a problem into parts. Abstraction *removes* unimportant details. Also, do not throw away details that really matter — "boil water" can never be ignored, but kettle brand can.

ایبسٹریکشن (Abstraction) کا مطلب ہے غیر ضروری تفصیلات کو چھپا کر صرف ضروری باتوں پر توجہ دینا۔ چائے بنانے کی مثال میں پانی ابلنا، پتی ڈالنا، دم دینا، کپ میں ڈالنا اور پیش کرنا ضروری مراحل ہیں؛ چائے کی قسم یا چینی متغیرات (Variables) ہیں؛ جبکہ کیتلی کا برانڈ یا کپ کا رنگ فالتو باتیں (Noise) ہیں جنہیں نظر انداز کیا جاتا ہے۔

CHECK YOURSELF

1. What is "noise" in abstraction?
2. In the tea example, is "sugar or lemon" a necessary step, a variable, or noise?
3. How is abstraction different from decomposition?

Answers: 1. Unnecessary details that do not affect the solution. 2. A variable (it changes with the situation). 3. Decomposition breaks a problem into parts; abstraction hides details that are not important.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Abstraction means:

- a. Hiding extra detail, keeping what matters
- b. Adding every detail
- c. Sorting slowly
- d. Drawing 7 OSI layers always

MCQ 2. A metro map that is not to scale is:

- a. Abstraction
- b. A truth table
- c. A histogram
- d. A key

MCQ 3. In coding, a function header without inner code is:

- a. Abstraction of how
- b. Decomposition only
- c. A race condition
- d. NAND

MCQ 4. Too little abstraction:

- a. Overwhelms you with noise
- b. Always speeds Big O
- c. Creates XOR
- d. Fixes phishing

MCQ 5. Too much abstraction:

- a. Hides a detail you actually need
- b. Prints SQL
- c. Draws K-maps
- d. Creates sprints

MCQ 6. A data type (list, stack) is an abstraction of:

- a. How items are stored/accessed
- b. TCP ports
- c. Screen DPI
- d. Figma frames

Extra short questions

1. Define abstraction.
2. Give a transport example.
3. Give a programming example.
4. How is abstraction different from decomposition?

Extra long question

1. For a college library app, list five details to keep and five to hide. Justify each as abstraction.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Hide unnecessary features; keep the essential ones needed to solve the problem.
2. Bus timetable shows stops and times, not engine temperature.
3. You call `sort(list)` without writing the swap loop.
4. Decomposition splits; abstraction hides. You often do both.

Long

1. Keep: ISBN, availability, due date, member id, fine. Hide: shelf paint colour, server fan speed, librarian's tea break, exact disk blocks, CSS pixels. Link to a simple interface of search/issue/return.